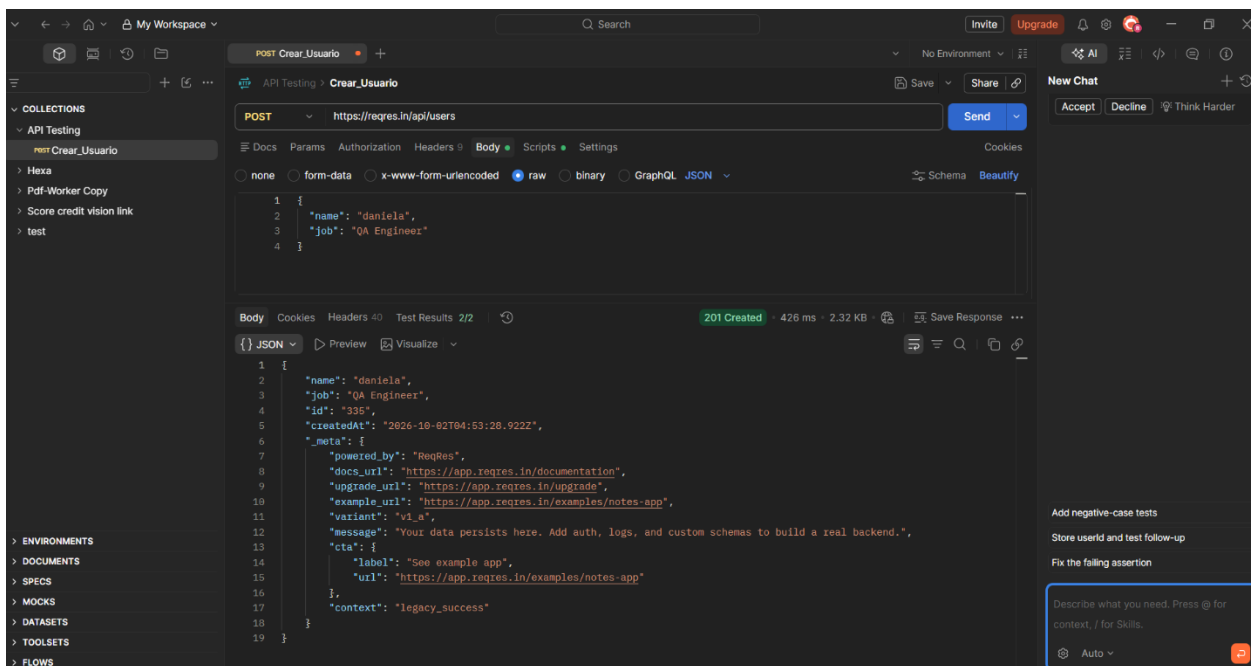


Informe de Evidencias de Pruebas Automatizadas de API

1. Creación de Usuario (POST /api/users)

Petición y Respuesta HTTP

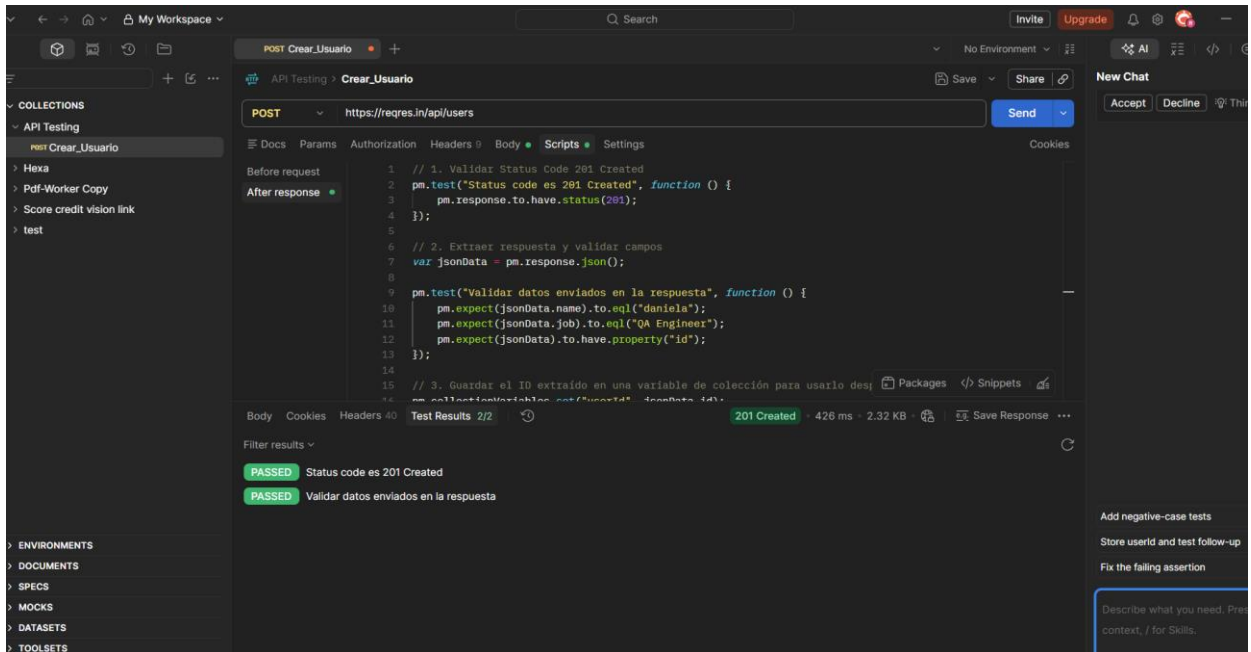


Objetivo: Validar la creación exitosa de un nuevo usuario en la plataforma mediante el envío de un payload JSON en el cuerpo de la solicitud.

Ejecución: Envíe una solicitud POST a /api/users enviando en el Body los atributos name: "daniela" y job: "QA Engineer".

Respuesta: El servidor retornó el código de estado 201 Created, devolviendo la confirmación de los datos enviados junto con el ID asignado (335) y la marca de tiempo de creación (createdAt).

Automatización y Aserciones



Objetivo: Automatizar la validación de la respuesta HTTP, verificar la estructura de datos y capturar el ID generado para futuras consultas.

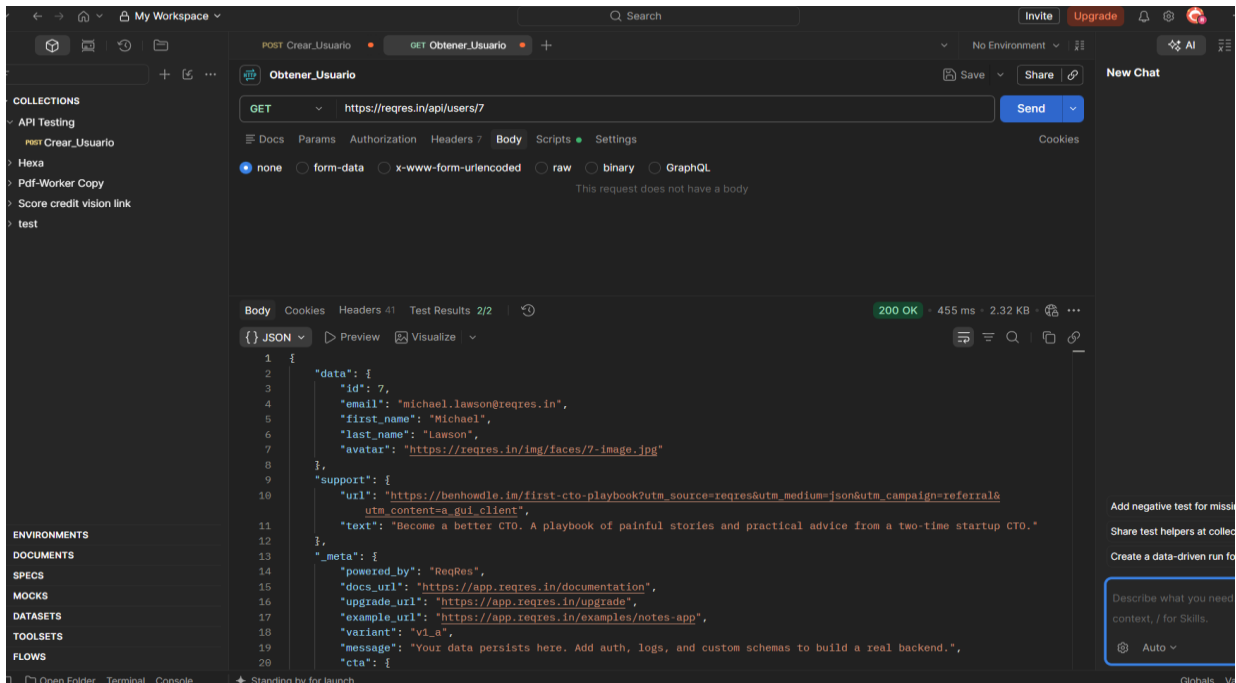
pm.response.to.have.status (201): Confirma que la API responda con el código 201 Created esperado.

pm.expect(): Valida que la respuesta contenga exactamente los valores enviados en name y job, y que incluya la propiedad id.

Gestión de estado: Se captura dinámicamente el valor de id para guardarlo en la variable de colección userId.

2. Consulta de Usuario (GET /api/users/{id})

Petición y Respuesta HTTP

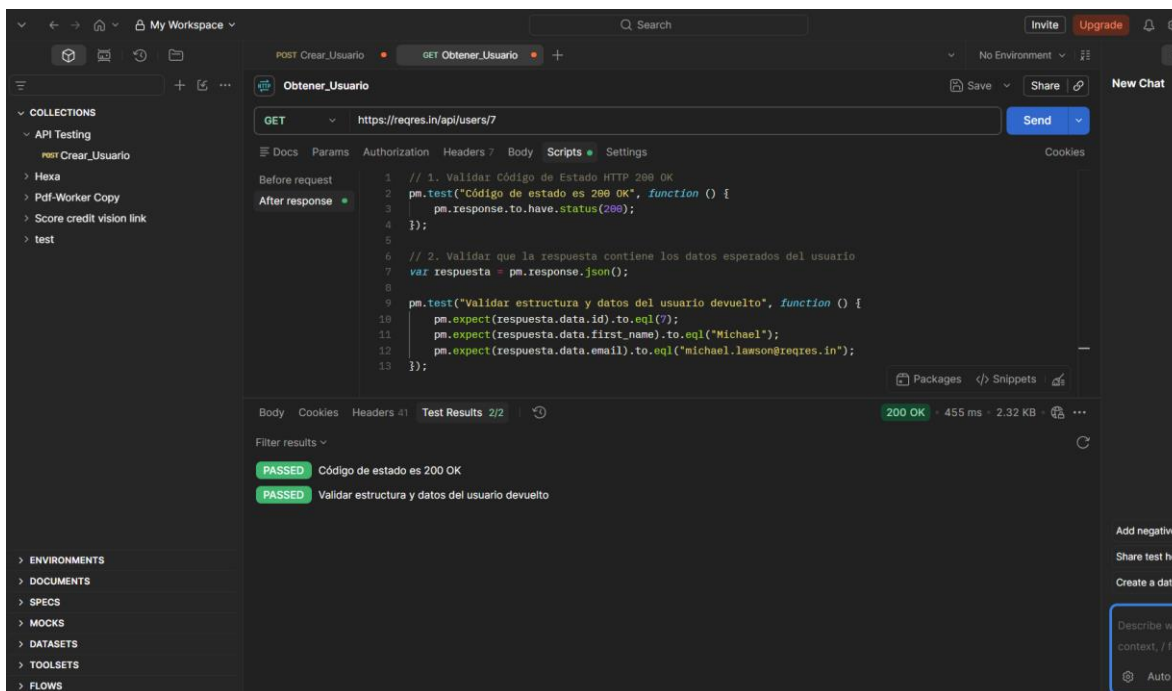


Objetivo: Consultar la información detallada de un usuario específico en el sistema para verificar la lectura de datos.

Ejecución: Envío una solicitud GET al endpoint `/api/users/7` para obtener la información de un usuario en la API.

Respuesta: La API respondió satisfactoriamente con el código 200, retornando la estructura JSON correspondiente dentro del objeto data id, email, first_name etc.

Automatización y Aserciones

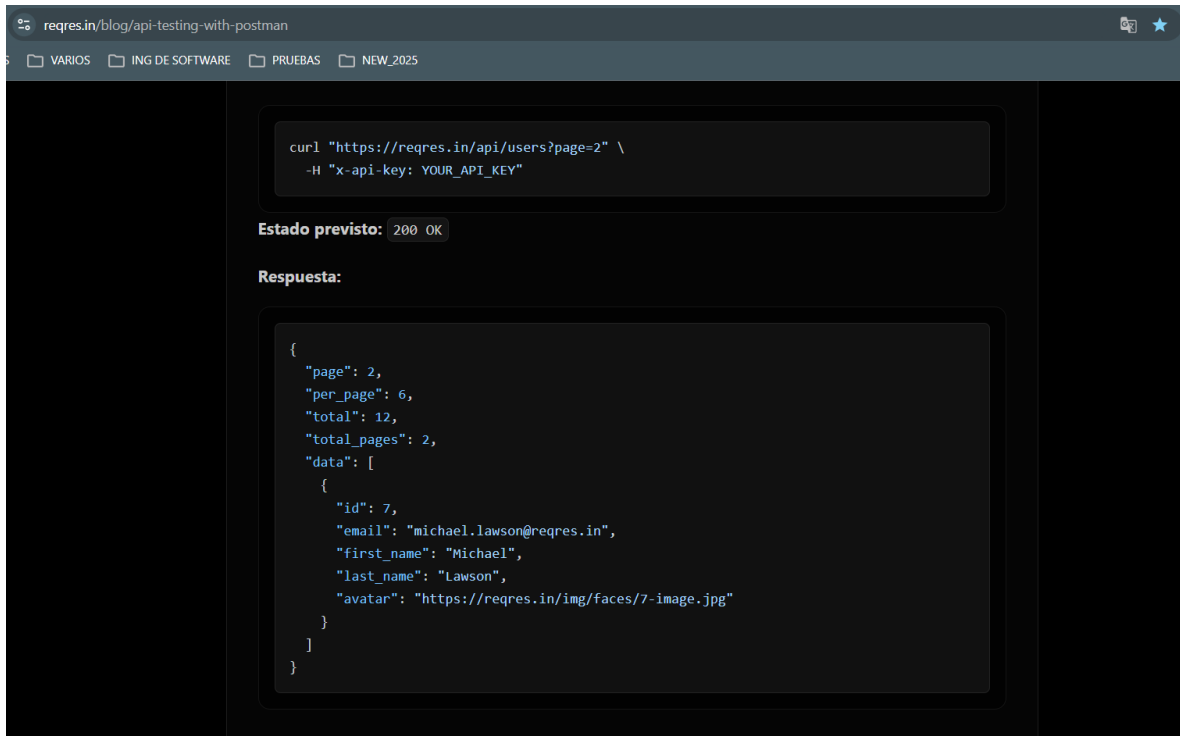


Objetivo: Verificar la precisión de los datos retornados por el endpoint y asegurar la estabilidad de la respuesta.

pm.response.to.have.status(200): Valida que el código HTTP sea 200 OK.

pm.expect(): Confirma que el objeto data contenga los campos esperados y los tipos de datos correctos para el usuario consultado (Michael con ID 7).

QA Note: Debido a que ReqRes es una API simulada de solo lectura y los registros enviados no persisten en una base de datos real, consulte el usuario Michael con el ID 7, que ya está quemado en la API para asegurar la respuesta 200 OK y validar la estructura real del JSON.



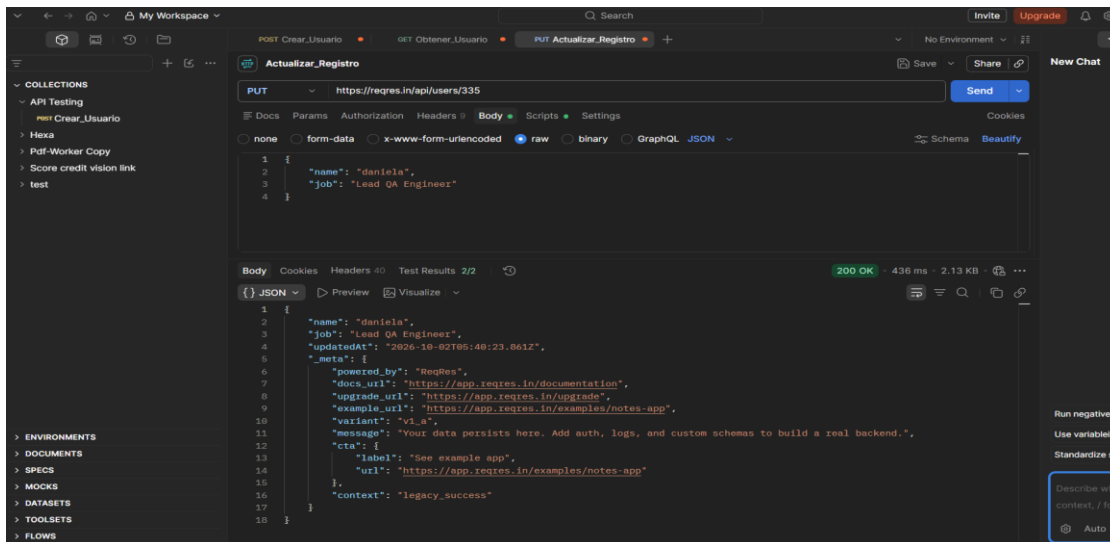
3. Actualización de Usuario (PUT /api/users/{id})

Petición y Respuesta HTTP

Objetivo: Validar la actualización de los atributos de un usuario existente.

Ejecución: Envíe una solicitud PUT al endpoint /api/users/335 modificando el cargo a Lead QA Engineer y manteniendo el nombre daniela.

Respuesta: La API retornó un código de estado 200 OK, devolviendo el JSON con la información actualizada y seteando el campo de tiempo updatedAt.

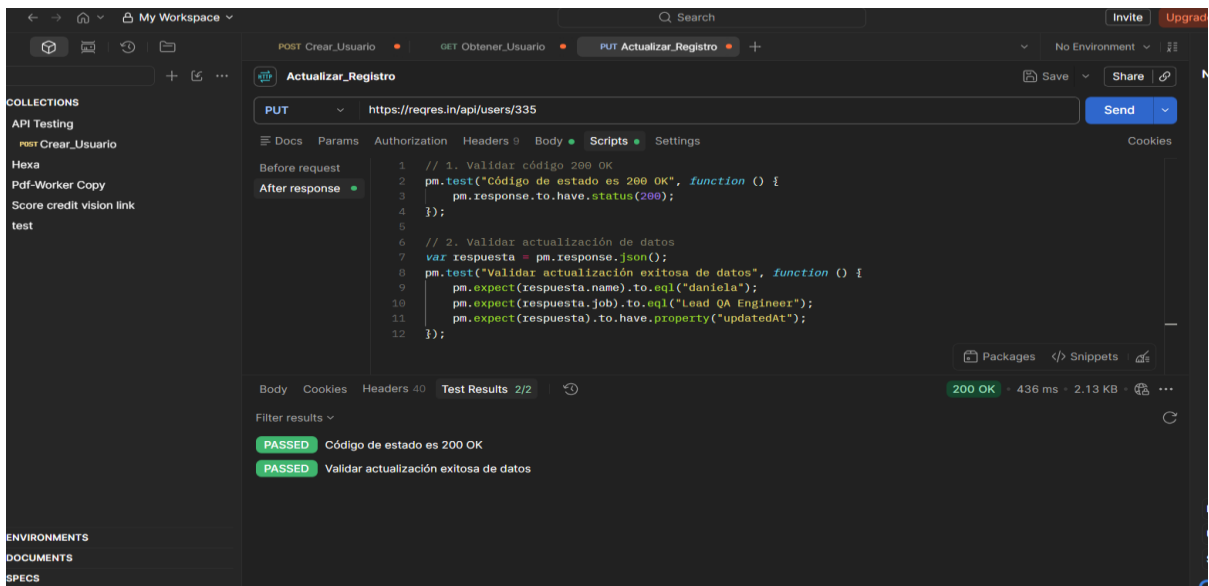


Automatización y Aserciones

Objetivo: Automatizar la verificación de los datos devueltos por la API posterior a la actualización de datos.

pm.response.to.have.status(200): Confirma la recepción del código 200 OK.

pm.expect(): Validar que los campos name y job reflejen los nuevos datos enviados y que exista la propiedad de auditoría updatedAt en el body.



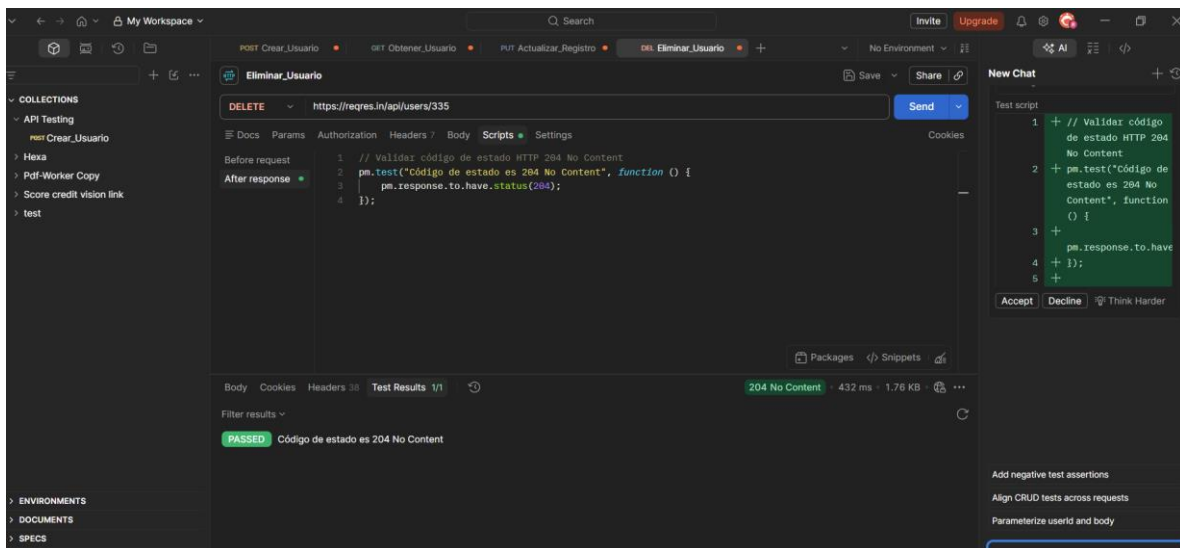
4. Eliminación de Usuario (DELETE /api/users/{id})

Petición y Respuesta HTTP

Objetivo: Verificar la correcta eliminación de un registro de usuario en el sistema.

Ejecución: Envíe una solicitud DELETE al endpoint /api/users/335 para solicitar el borrado del usuario.

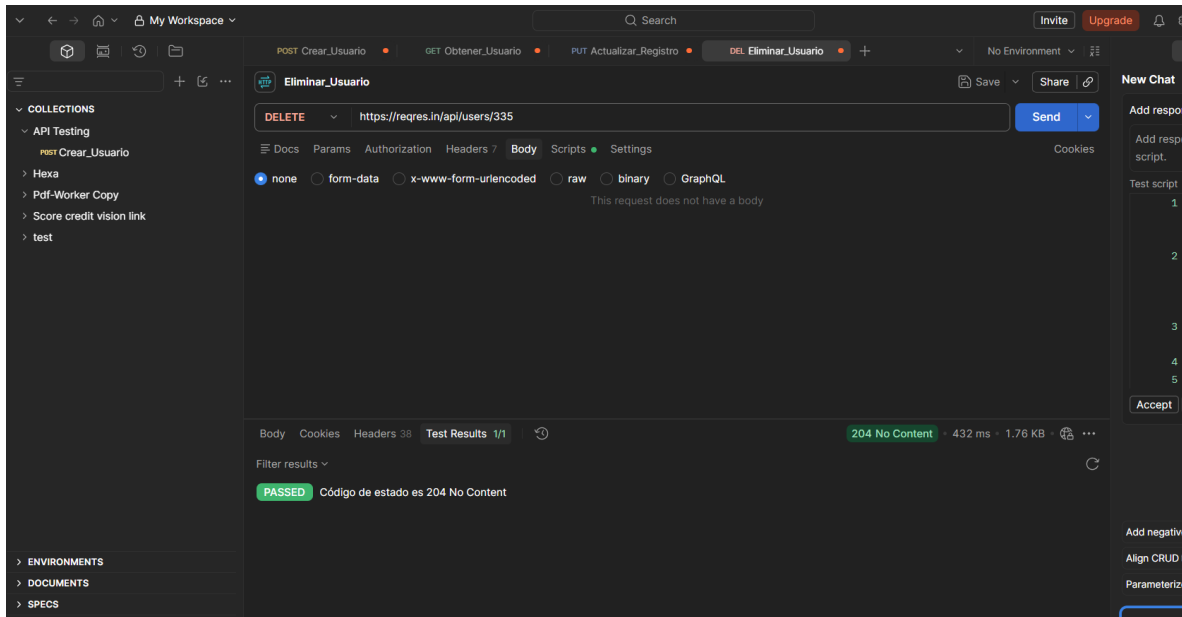
Respuesta: El servidor respondió con el código de estado 204 No Content, indicando que la operación de eliminación fue exitosa y sin retornar cuerpo de datos, conforme al estándar REST.



Automatización y Aserciones

Objetivo: Validar de forma automatizada que el servidor confirme la eliminación con el código HTTP estándar para solicitudes sin cuerpo de respuesta.

pm.response.to.have.status(204): Garantiza que la API retorne exactamente el código 204 No Content tras procesar la eliminación.



5. Prueba Negativa: Usuario No Encontrado (GET /api/users/23)

Petición y Respuesta HTTP

Objetivo: Validar el comportamiento defensivo de la API ante solicitudes de recursos inexistentes.

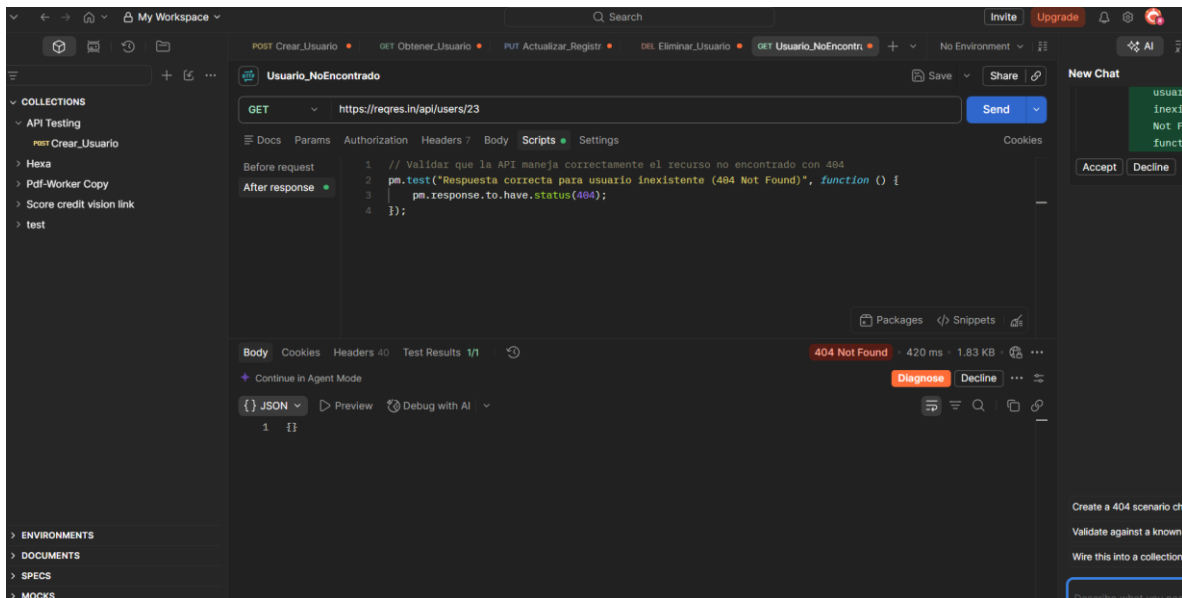
Ejecución: Envíe una solicitud GET consultando un identificador inexistente (/api/users/23).

Respuesta: La API gestionó el caso borde adecuadamente respondiendo con el código de estado 404 Not Found y el body vacío {}.

Automatización y Aserciones

Objetivo: Automatizar la verificación del manejo de errores y validación de límites en el backend.

pm.response.to.have.status(404): Confirma que la API informe de manera precisa la ausencia del recurso mediante el código de estado 404 Not Found.



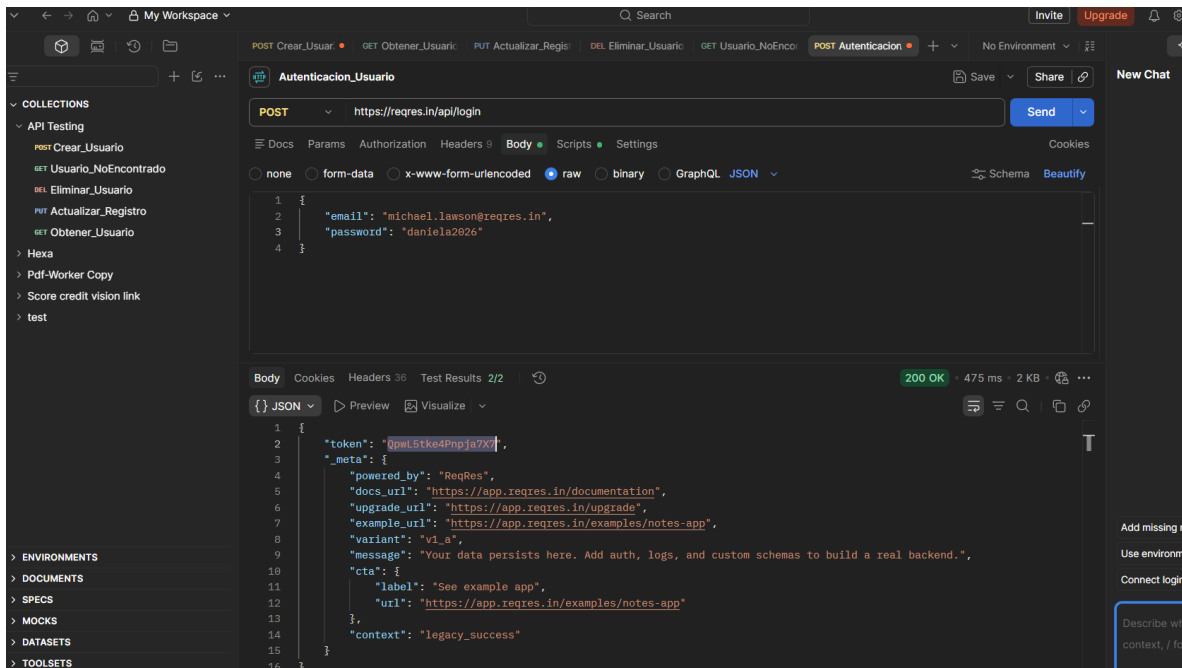
6. Autenticación de Usuario (POST /api/login)

Petición y Respuesta HTTP

Objetivo: Validar el flujo de autenticación de usuarios y la generación del token de seguridad para el acceso a endpoints protegidos.

Ejecución: Envíe una solicitud POST a /api/login con credenciales válidas en el cuerpo JSON (email y password), en este caso utilice el email del usuario utilizado anteriormente, michael

Respuesta: La API autenticó al usuario retornando el código 200 OK junto con el token de acceso generado - QpwL5tke4Pnpja7X7



Automatización y Aserciones

Objetivo: Automatizar la validación del protocolo de autenticación y asegurar que el token devuelto cumpla con los requisitos mínimos de formato.

pm.response.to.have.status(200): Valida la recepción de la confirmación 200 OK.

pm.expect(): Verifica que el JSON contenga la propiedad token y que esta sea una cadena de texto no vacía.

